BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: C Batch: 2 Practical No: 3

Roll no: 56 PRN No: 12411305 Name of Student: Anuja Dhiraj Kumbhar

Subject Name : Deep Learning

Problem Statement :

Implement forward propagation and backpropagation using TensorFlow/Keras. Analyze the effect of different learning rates and the number of epochs on model performance.

Algorithm :

1. Start the program.
2. Import the required libraries.
3. Load and preprocess the dataset.
4. Create the neural network model.
5. Compile the model using Adam optimizer.
6. Train the model with different learning rates and epochs.
7. Evaluate the model on the test data.
8. Display accuracy and plot the graphs.
9. Stop the program.

Code :

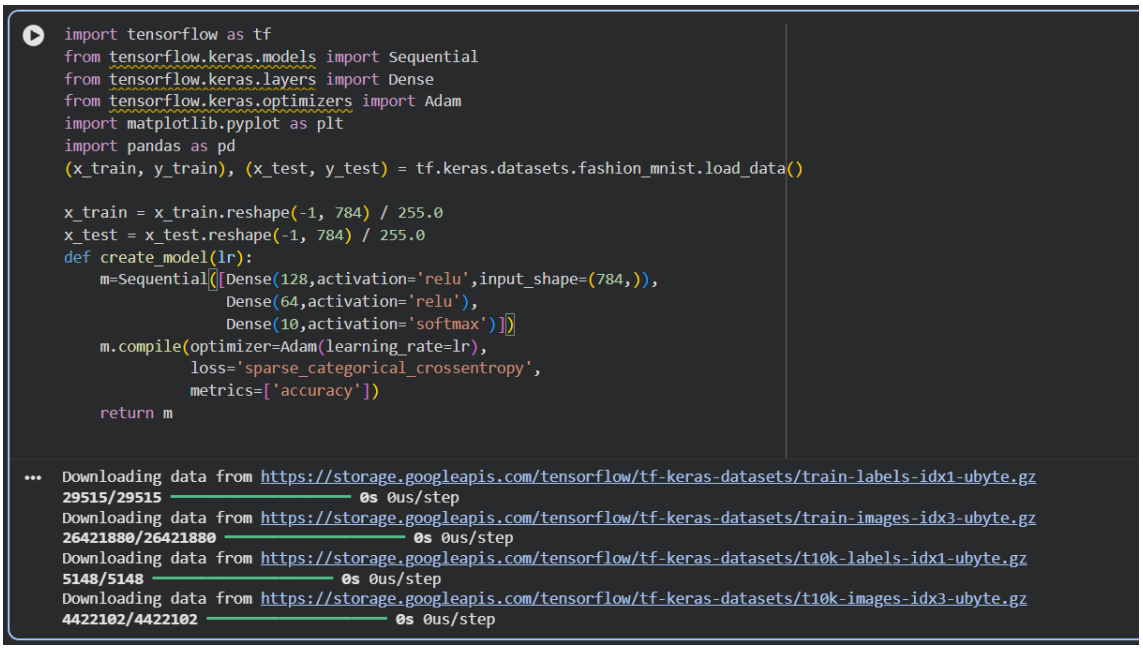
```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam
import matplotlib.pyplot as plt
import pandas as pd
(x_train, y_train), (x_test, y_test) =
tf.keras.datasets.fashion_mnist.load_data()

x_train = x_train.reshape(-1, 784) / 255.0
x_test = x_test.reshape(-1, 784) / 255.0
def create_model(lr):
    m=Sequential([Dense(128,activation='relu',input_shape=(784,)),
                  Dense(64,activation='relu'),
                  Dense(10,activation='softmax')])
    m.compile(optimizer=Adam(learning_rate=lr),
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
    return m
learning_rates=[0.1,0.01,0.001]
results=[]
for lr in learning_rates:
    model=create_model(lr)
    model.fit(x_train,y_train,epochs=5,batch_size=128,verbose=0)
    loss,acc=model.evaluate(x_test,y_test,verbose=0)
    results.append([lr,acc])
df=pd.DataFrame(results,columns=['Learning Rate','Accuracy'])
print(df)
plt.plot(df['Learning Rate'],df['Accuracy'],marker='o')
plt.xscale('log'); plt.grid(); plt.show()
epochs_list=[5,10,20]
res=[]
for ep in epochs_list:
    model=create_model(0.001)
    h=model.fit(x_train,y_train,validation_split=0.2,epochs=ep,batch_size=
128,verbose=0)
    loss,acc=model.evaluate(x_test,y_test,verbose=0)
    res.append([ep,acc])
edf=pd.DataFrame(res,columns=['Epochs','Accuracy'])
print(edf)
plt.plot(edf['Epochs'],edf['Accuracy'],marker='o')
plt.grid(); plt.show()
```

```
model=create_model(0.001)
history=model.fit(x_train,y_train,validation_split=0.2,epochs=20,batch_siz
e=128)
plt.plot(history.history['accuracy']);
plt.plot(history.history['val_accuracy']); plt.legend(['Train','Val']);
plt.show()
plt.plot(history.history['loss']); plt.plot(history.history['val_loss']);
plt.legend(['Train','Val']); plt.show()
print(model.evaluate(x_test,y_test))
```

Output :

- **Model creation and loading dataset**



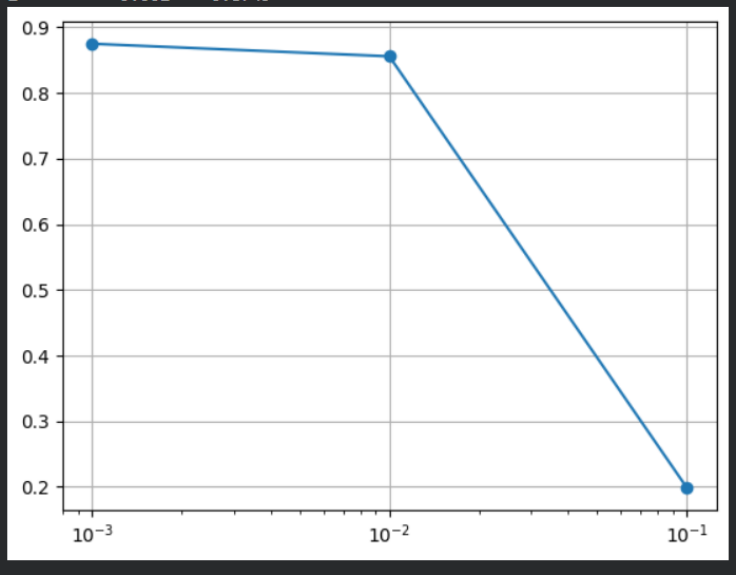
```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam
import matplotlib.pyplot as plt
import pandas as pd
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.fashion_mnist.load_data()

x_train = x_train.reshape(-1, 784) / 255.0
x_test = x_test.reshape(-1, 784) / 255.0
def create_model(lr):
    m=Sequential([Dense(128,activation='relu',input_shape=(784,)),
                  Dense(64,activation='relu'),
                  Dense(10,activation='softmax')])
    m.compile(optimizer=Adam(learning_rate=lr),
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
    return m
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz
29515/29515 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz
26421880/26421880 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz
5148/5148 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz
4422102/4422102 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
```

- **Learning rate comparison**

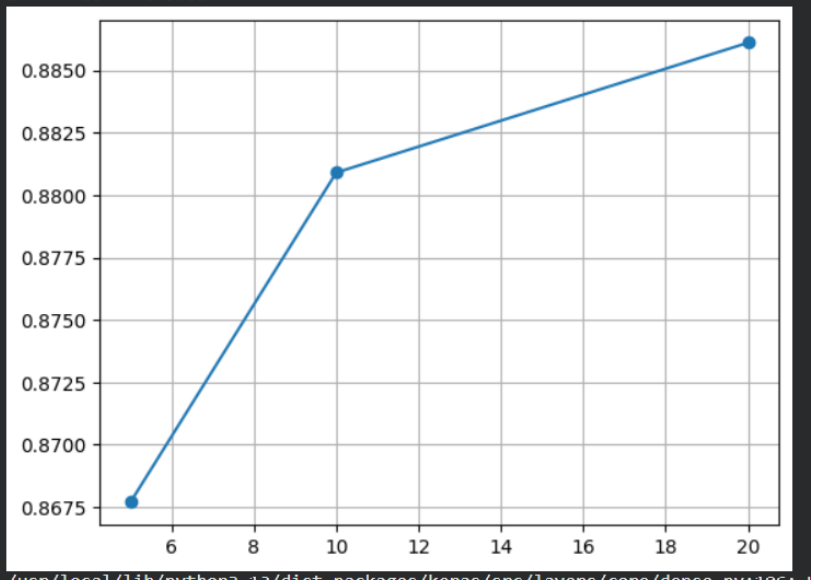
```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserW
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
   Learning Rate  Accuracy
0          0.100    0.1992
1          0.010    0.8557
2          0.001    0.8749
```



- **Epoch-wise Accuracy Analysis**

```
print(model.evaluate(x_test,y_test))
```

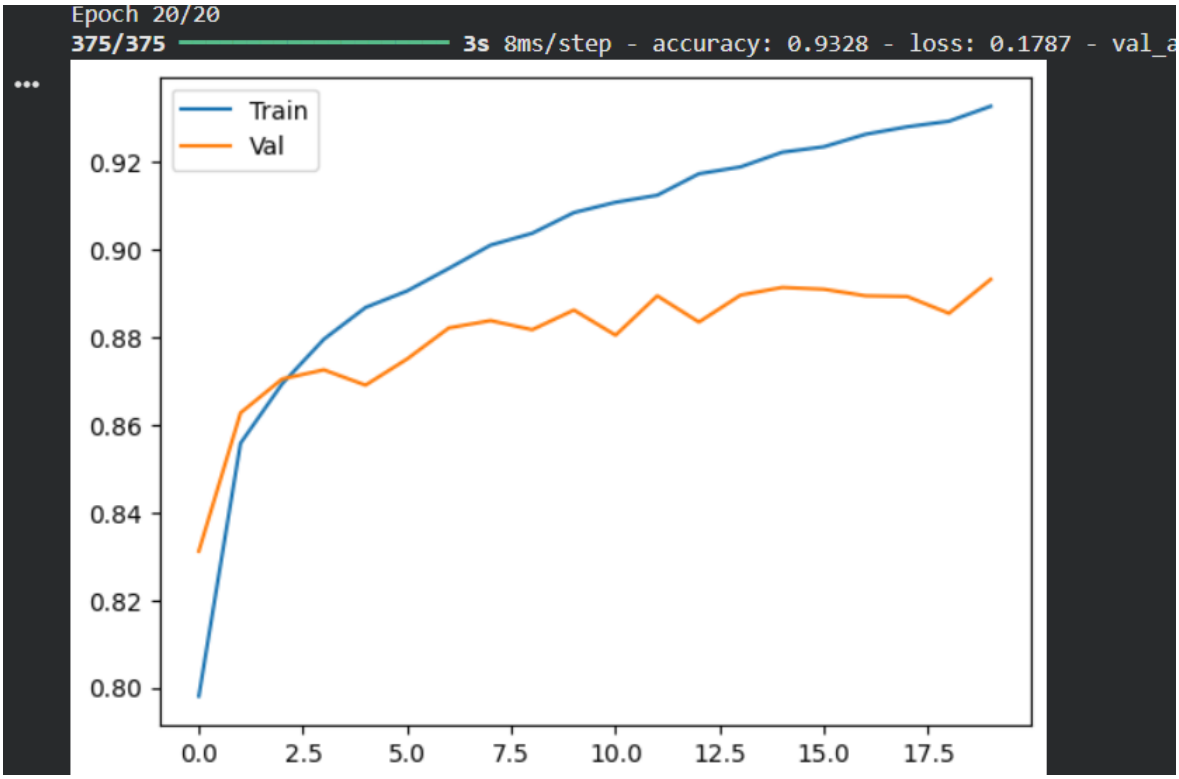
```
   Epochs  Accuracy
0       5    0.8677
1      10    0.8809
2      20    0.8861
```



- **Model training progress**

```
plt.plot(history.history['loss']); plt.plot(history.history['val_loss']); plt.legend(['Train','Val']); plt.show()
print(model.evaluate(x_test,y_test))
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.8868 - loss: 0.3107 - val_accuracy: 0.8691 - val_loss: 0.3601
Epoch 6/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.8906 - loss: 0.2970 - val_accuracy: 0.8751 - val_loss: 0.3441
Epoch 7/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.8958 - loss: 0.2830 - val_accuracy: 0.8822 - val_loss: 0.3267
Epoch 8/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.9011 - loss: 0.2698 - val_accuracy: 0.8838 - val_loss: 0.3225
Epoch 9/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - accuracy: 0.9038 - loss: 0.2605 - val_accuracy: 0.8817 - val_loss: 0.3349
Epoch 10/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - accuracy: 0.9085 - loss: 0.2501 - val_accuracy: 0.8863 - val_loss: 0.3169
Epoch 11/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.9109 - loss: 0.2405 - val_accuracy: 0.8805 - val_loss: 0.3284
Epoch 12/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.9125 - loss: 0.2345 - val_accuracy: 0.8895 - val_loss: 0.3095
Epoch 13/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.9174 - loss: 0.2231 - val_accuracy: 0.8835 - val_loss: 0.3264
Epoch 14/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.9190 - loss: 0.2165 - val_accuracy: 0.8897 - val_loss: 0.3183
Epoch 15/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 4s 10ms/step - accuracy: 0.9223 - loss: 0.2086 - val_accuracy: 0.8914 - val_loss: 0.3183
Epoch 16/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.9236 - loss: 0.2045 - val_accuracy: 0.8910 - val_loss: 0.3174
Epoch 17/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.9264 - loss: 0.1968 - val_accuracy: 0.8895 - val_loss: 0.3268
Epoch 18/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.9281 - loss: 0.1920 - val_accuracy: 0.8893 - val_loss: 0.3278
Epoch 19/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.9294 - loss: 0.1887 - val_accuracy: 0.8855 - val_loss: 0.3405
```

- **Training vs Validation Accuracy**



- **Training vs Validation Loss**